



PREPARED FOR:
Company: **EPI-USE Africa**
Phone: **(816) 309-6795**
Mail: **mail@email.com**

PREPARED BY:
Name: **CDT**
Mail: **capstonedevteam@gmail.com**

AutoFlow Trading Project Tender

2026





Project Description

AutoFlow provides an interactive learning experience by providing a virtual trading and investments environment that uses real market data to expose users to real world volatility safely. It focuses on short term, higher risk investments such as stocks, options, cryptocurrencies, and commodities to keep users engaged and highlight risk.

Autoflow also features hands-on experience with risk mitigation strategies such as conditional orders and custom notification rules tied to price changes.

CDT proposes expanded gamification features beyond a simple leaderboard, similar to that of Duolingo or fantasy sports games to increase user engagement and retention, which will be elaborated on below.

Background & Motivation

Existing trading platforms do not provide much guidance for new users, at most providing a single practice account with a fixed starting balance and leaving users to explore cluttered UIs and confusing metrics themselves. Autoflow seeks to ease inexperienced students and hobbyists into the world of financial markets in a safe environment and with optional guided learning.

System Requirements

We plan to fulfill all main and optional requirements, as well as the “AI Portfolio Investment Assistant” wow factor. They are repeated below for convenience:

- Real-Time Data Visualization
- View price changes for different timeframes: e.g., daily, weekly, monthly
- Share price & Daily high/low: Display key metrics
- Intuitive User Interface: Simple navigation, popular investments shown
- User Portfolio Management: Virtual account with initial funds
- Simulated Investments: Detailed log system for every trade
- Caching: Cache fetched stock information where allowed

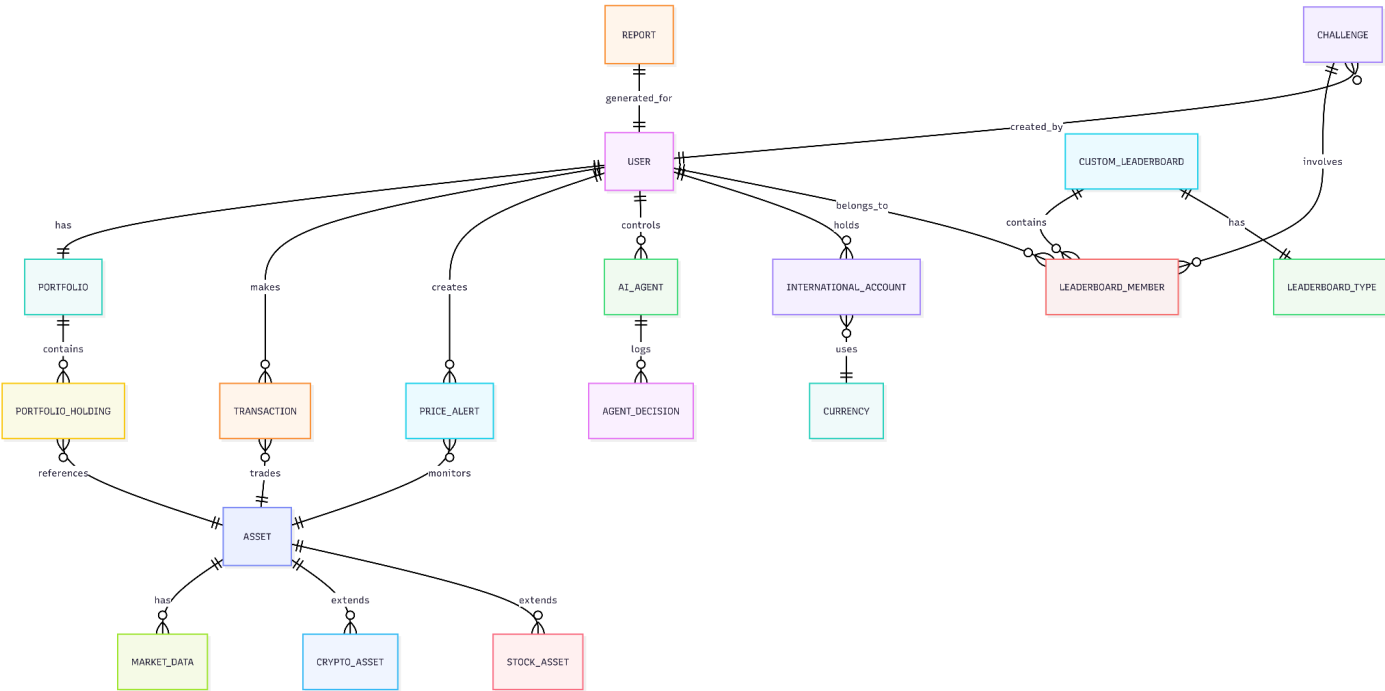
Optional Requirements

- Multiple International Accounts
- Daily/Weekly report on Stock Changes
- Cryptocurrencies added
- Buy/sell/notify when price hits a certain point
- Leaderboards

Wow Factors (client labeled as optional)

- AI Portfolio Investment Assistant
- AI Agent to trade for the user

Proposed Domain Model



Proposed Technologies

Layer	Technology
Frontend	Next.js
Frontend Styling	TailwindCSS
Frontend Hosting	Vercel (free)
Backend	FastAPI
Backend Hosting	Render (free → \$7 demo month)
API Style	REST
Real-time	WebSockets (FastAPI built-in)
Database	PostgreSQL (Supabase free)
Queue	Redis (Upstash free — or containerized Redis locally)
Background Tasks	Celery
AI Framework	Scikit-learn
Price Data	FCS API (free) + mock for options
Authentication	JWT
Load Handling	Gunicorn with multiple workers
Containerization	Docker + Docker Compose
Documentation	Swagger UI + ReDoc (automatic)
CI/CD	GitHub Actions
Version Control	GitHub

Next.js

We chose Next.js for its excellent performance, built-in routing, and seamless deployment on Vercel. It allows us to build a fast, responsive trading dashboard with minimal configuration.

TailwindCSS

TailwindCSS enables rapid UI development with utility-first styling. It keeps our CSS bundle small, ensures design consistency, and was recommended by the client.

Vercel

Vercel provides zero-configuration deployment for Next.js with automatic SSL and a global CDN. Its generous free tier is more than sufficient for our needs.

FastAPI

FastAPI offers native async support, automatic API documentation, and seamless integration with Python's AI ecosystem. Its performance and WebSocket support make it ideal for a real-time trading platform.

Render

Render provides simple deployment with automatic HTTPS and GitHub integration. Its free tier works for development, and a low-cost paid upgrade eliminates cold starts for our final demos and project day.

REST API

REST is simple, stateless, and familiar to our team. It integrates cleanly with FastAPI and provides intuitive endpoint organization for our trading operations.

WebSockets

WebSockets enable real-time price streaming and instant notifications. FastAPI's built-in WebSocket support allows us to push live data to clients without complex additional infrastructure.

PostgreSQL (Supabase)

PostgreSQL provides ACID compliance and reliable row-level locking for concurrent transactions. Supabase's free tier offers a managed database with automatic backups at no cost.

Redis (Upstash)

Redis serves as a fast, lightweight message broker for our Celery task queue. Upstash provides a managed free tier that meets our needs without self-hosting.

Celery

Celery handles long-running tasks like AI trading decisions in the background. This prevents blocking the main application thread and keeps the interface responsive.

Scikit-learn

Scikit-learn provides production-ready machine learning models for price prediction and trading signals. It is lightweight, well-documented, and native to our Python backend.

FCS API with Mock Options

FCS API covers stocks, crypto, and commodities on a generous free tier. For options data, we will use realistic mock data supplemented by a free trial during the demo period.

JWT Authentication

JWT provides stateless, cryptographically secure authentication. It is lightweight, widely adopted, and works well with our REST API architecture.

Gunicorn

Gunicorn allows us to run multiple worker processes on a single server, distributing load and improving concurrency. It also provides automatic worker recovery.

Docker

Docker ensures consistent environments across all team members' machines. It eliminates 'works on my machine' issues and simplifies deployment.

Swagger UI and ReDoc

FastAPI automatically generates interactive API documentation using Swagger UI and ReDoc. This provides testable, up-to-date documentation with zero maintenance.

GitHub Actions

GitHub Actions automates testing and deployment on every push. It catches issues early and reduces manual deployment overhead.

GitHub

GitHub is required by the course and provides version control, pull request workflows, and issue tracking. It integrates seamlessly with our CI/CD pipeline.

Constraints

The project operates under several constraints. API rate limits from free-tier market data providers restrict update frequency, mitigated by a fallback to realistic mock data. Enterprise-grade alternatives are cost-prohibitive, so we prioritize open-source solutions and reserve the R1000 budget for strategic one-time upgrades. As specified in the original proposal, the system must not suffer from slippage, must respect API terms of use when caching, and must not use external LLM APIs — all AI agents will be built from scratch. Free-tier hosting limitations include memory caps and cold starts, acceptable during development and eliminated by a paid upgrade during the demo month.

Budget

Scenario	Polygon.io	Render Paid	Custom Domain	T-Shirts Banner	Total
Full tech + minimal marketing spend	R537 (1 month)	R130 (1 month)	R100 (1 year)	R233	R1,000
Tech + Marketing	R0 (use free trial)	R130 (1 month)	R100 (1 year)	R770	R1,000
Minimal tech + Max Marketing	R0 (mock only)	R0 (free tier)	R0 (skip)	R1,000	R1,000

Development Methodology: Scrumban

We chose Scrumban (Scrum + Kanban) because this project requires predictable delivery, mindfulness of rate limits and quick response times to resolve issues. We found that neither Scrum nor Kanban alone satisfies all requirements.

Requirement	How Scrumban Delivers
Predictable delivery	Two-week sprints with a single goal. You see working software every iteration.
API rate limit discipline	WIP limits cap concurrent API-dependent tasks (e.g., max 2). Prevents overload.
Bottleneck resolution	A Project Board shows code review or testing pileups instantly. We swarm to clear them.
Emergency fixes	Expedite lane (WIP=1) for critical API failures. No sprint interruption.

Quality completion	&	Definition of Done includes caching, logging, testing. No half-finished features.
Team of five		Clear roles (CR, SM, three developers). WIP limits matched to team size.

Scrum gives us planning and demos. Kanban gives us flow control and rate limit protection. Together, they deliver your stock simulator on time, within API constraints, and minimising development bottlenecks.

Our Team – Todo: Designate CR and SM

Michael Neto

Github link: <https://github.com/u24641342>

LinkedIn: <https://www.linkedin.com/in/michael-neto-0211563b9/>

Michael is a full-stack developer studying Information and Knowledge Systems. His strengths lie in learning quickly and solving problems. He has experience in working with teams and a skill set suited for extensive web application development. His passion extends beyond just front-end development and design, and into catering and improving the end-user's experience.

My top programming languages/frameworks are:

Javascript

ReactJS

Angular

Java

C++

Dillon Kung

Github link: <https://github.com/u24614582>

Dillon is a developer studying his final year of computer science. He enjoys problem solving and acquiring new skills. He has a wide range of knowledge pertaining to web development and has experience working with large teams. Dillon has an interest in back-end business logic and database design. He takes pride in contributing to his team and providing solutions that are both elegant and effective.

Top programming languages/frameworks:

C++

Java

Javascript

NestJS

SQL

Caitanya Singh

Github link: <https://github.com/u24603199>

LinkedIn:

<https://www.linkedin.com/in/caitanya-krpa-narain-singh-4316393b6/>

Caitanya is a backend developer with some experience in designing and implementing scalable backend architectures. He also has extensive knowledge of developing and integrating API's using a variety of technologies, as well as deploying applications via Docker. Caitanya has a passion for building robust backend systems and well-structured databases, as demonstrated through his academic work and personal projects

Top Technologies

MongoDB

Docker

PHP

Node.js & express

Typescript

Grant Nel

Github link: <https://github.com/u24574547>

LinkedIn: <https://www.linkedin.com/in/grant-nel-72a0573b5>

Grant is a 3rd year Computer Science student with an interest in mathematics and financial markets alongside his passion for working on complex systems. His interest in niche Computer Science fields such as novelty operating systems and archaic devices have afforded him some uncommon proficiencies in networking and problem solving compared to his peers alongside proficiency in industry standard desktop and mobile technologies.

He has experience in developing applications for small businesses, lending him insight into practical challenges faced by end-users outside of the academic context.

Top technologies:

Java

Typescript (React, Next.js, NestJS, Angular, Ionic)

C++

SQL (Microsoft SQLServer, MySQL, PostgreSQL)

PHP

Finnley Wyllie

Github link: <https://github.com/Finn-Wyll>

LinkedIn: <https://za.linkedin.com/in/finnley-wyllie-1b24972a7>

Finnley is a final- year Computer Science student specialising in backend systems and software engineering. He has a genuine passion for dissecting complex problems and crafting solutions that are efficient and elegant. He has hands-on experience building secure APIs, designing database architectures and developing robust backend solutions. His proficiency in C++, Go and Typescript highlight a preference for precision, performance and clean engineering. As part of this team, Finnley brings the technical depth and curiosity to ensure what is built is functional, efficient and well-crafted.

Core Technologies :

C++

Python

Go

Javascript/ Typescript

PHP

Additional Skills:

Machine Learning & Model Training

Computer Vision

Information Theory and Probabilistic Reasoning

THANK YOU

Image Placeholders

• [Logo of EPI-USE Africa]